

DEVOPS E ARQUITETURA CLOUD

FASE 3 | AULA 01 -

FUNDAMENTOS: DO GIT AO PIPELINE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	12
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

O QUE VEM POR AÍ?

Em um mundo onde a velocidade de *deployment* é crítica, o pipeline de Integração e Entrega Contínua (CI/CD) se tornou o epicentro estratégico da entrega de valor, mas também a nova superfície de ataque da cadeia de suprimentos de software. Nesta aula, transcendemos a visão básica do DevOps para mergulhar na Engenharia de Pipelines Seguros, adotando a mentalidade DevSecOps para garantir que cada *commit* não apenas seja rápido, mas fundamentalmente seguro. Nossa jornada é de arquitetura, focada em fechar vetores de ataque e construir resiliência desde o início, garantindo que o seu sistema, como postulado pela Engenharia de Confiabilidade de Sites (SRE), seja intrinsecamente confiável.

HANDS ON

A experiência prática será o ponto focal desta aula. Para que os conceitos de segurança se tornem sólidos, será imperativo vivenciar o risco. O laboratório guiará você pela construção de um *workflow* inicial no GitHub Actions, onde intencionalmente configuraremos um anti-padrão: o uso de credenciais de longa duração, as Access Keys da AWS, para realizar o *deploy*. Em seguida, simularemos uma exploração, demonstrando de forma prática o impacto catastrófico do vazamento dessas chaves na sua conta AWS, expondo o vetor de ataque conhecido como Higiene Insuficiente de Credenciais (CICD-SEC-6).

Este exercício prático visará estabelecer uma base de conhecimento profundo sobre porque a migração para a Federação de Identidade de Workload (OIDC) é um requisito arquitetural. Ao interagir com o código disponibilizado no Github, você poderá diagnosticar a falha, entender a causa raiz e, em aulas futuras, refatorar o pipeline para um modelo de confiança zero.

SAIBA MAIS

A Fundação Estratégica: Secure SDLC e a Economia do Shift-Left

O DevSecOps representa a maturidade do ciclo de vida de desenvolvimento de software (*Software Development Lifecycle* - SDLC), incorporando cultura, automação e design de segurança em todas as suas fases. Este movimento é o reconhecimento de que, em ambientes de entrega contínua, a segurança não pode ser um *gateway* de aprovação tardio, mas sim um componente intrínseco, aplicado em cada *commit* e revisão de código. A segurança é transformada de um processo de conformidade para um fator de qualidade do produto.

O pilar central dessa transformação é o **mandato do Shift-Left**, que exige que as atividades de segurança sejam deslocadas para as fases iniciais do SDLC—desde o design e codificação, alinhando-se estritamente aos princípios DevOps. A filosofia subjacente é que quanto mais cedo uma vulnerabilidade é identificada e mitigada, menor será seu impacto e custo de correção.

Para líderes de projetos focados em otimização de processos e finanças, o *Shift-Left* é, fundamentalmente, uma **estratégia de otimização de custo e produtividade**. A análise do impacto financeiro reforça essa necessidade: corrigir uma falha de segurança detectada após a aplicação estar em produção pode ser, em muitas situações, de **100 a 640 vezes mais caro** do que resolvê-la durante a fase de design ou codificação. A detecção precoce de vulnerabilidades na fase de desenvolvimento, em vez de esperar pelos testes finais, evita o acúmulo de débito técnico (*technical debt*) significativo e reduz a necessidade de *panicked remediation moments*. Corrigir um defeito durante a fase de implementação pode levar apenas cerca de trinta minutos, em contraste com as quinze horas exigidas para a mesma correção se o defeito for descoberto somente em produção. Portanto, o *Shift-Left* deve ser visto como um imperativo financeiro e de eficiência operacional. Sua implementação madura permite a realocação de milhares de horas de engenharia da remediação para a inovação. Essa perspectiva enquadra a segurança não como um custo, mas como uma ferramenta de otimização de produtividade.

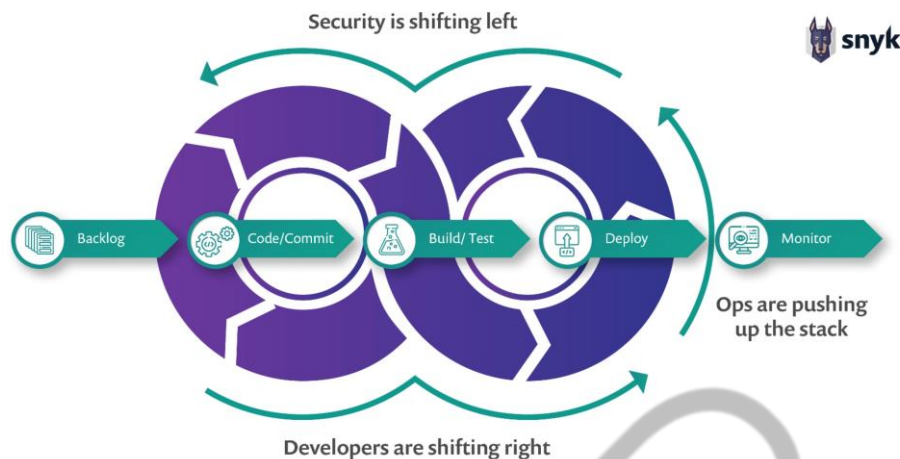


Figura 1 - DevSecOps Shift-Left
Fonte: [Cloudinary](#) (2025)

Modelagem de Ameaças (Threat Modeling) no CI/CD

A **Modelagem de Ameaças** é o processo proativo de identificação sistemática de ativos, fronteiras de confiança e fluxos de dados dentro de uma arquitetura, com o objetivo de mapear o que pode dar errado antes que o código entre no pipeline. Em um contexto de CI/CD, a Modelagem de Ameaças deve ser um passo mandatório nas revisões de design de qualquer novo pipeline ou serviço, definindo domínios de privilégio e classificações de dados.

A metodologia **STRIDE** (*Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege*) fornece o framework mais estruturado para esta análise. Ao aplicar o STRIDE aos componentes críticos do pipeline — como o *Source Control Management* (SCM), os *runners* e os repositórios de artefatos — os engenheiros podem antecipar e mitigar os vetores de ataque de forma sistemática.

A categoria **Tampering** (Violação), por exemplo, foca nas ameaças à integridade do código e dos artefatos de *build*. Isso leva à mitigação fundamental através da **assinatura e validação de artefatos**, garantindo que o que foi construído é exatamente o que está sendo implantado. Já a categoria **Elevation of Privilege** (Escalação) está diretamente ligada ao gerenciamento de acesso (IAM) e aos privilégios concedidos aos *runners* de CI/CD, exigindo o princípio de *Least Privilege* e o uso de tokens de curta duração.

A Modelagem de Ameaças também se conecta aos pilares de engenharia de confiabilidade (*Site Reliability Engineering* - SRE). O *Google SRE Book* enfatiza o tripé Confidencialidade, Integridade e Disponibilidade (CIA). A segurança do pipeline deve reforçar primariamente a **Integridade** (protegendo o código contra adulteração, o que é mitigado por assinaturas de *builds*) e a **Disponibilidade** (protegendo contra ataques que visam a saturação de recursos dos *runners*). Dada a natureza dos atacantes modernos, que utilizam automação e inteligência artificial para explorar sistemas, a modelagem de ameaças deve ser contínua e automatizada, transformando o modelo em um *framework* executável dentro do pipeline.

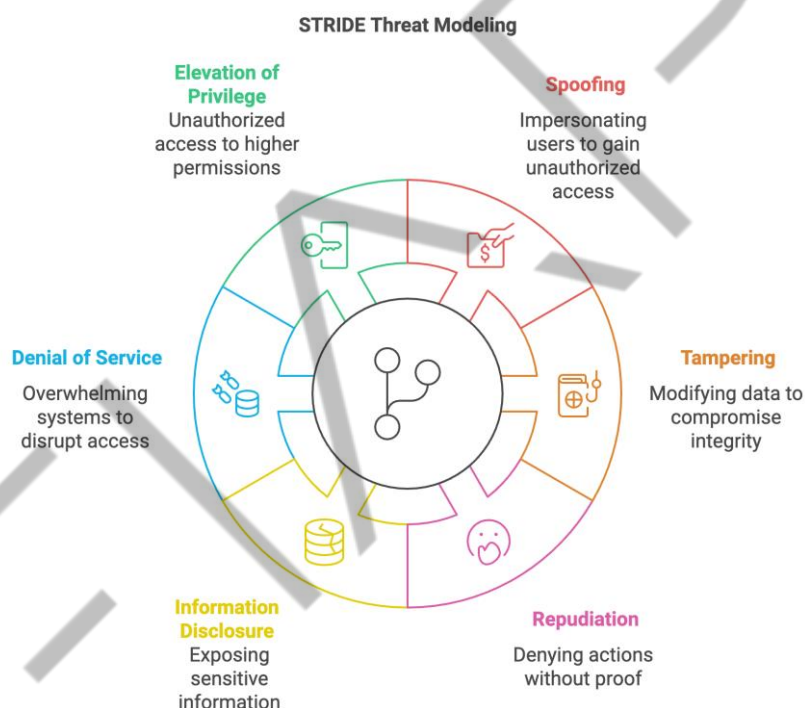


Figura 2 – DevSecOps STRIDE
Fonte: [Jetbrains](#) (2025)

Análise de Vetores de Ataque: O OWASP Top 10 CI/CD e a Mitigação

O projeto OWASP CI/CD Security fornece uma taxonomia padrão de dez riscos críticos que afetam os pipelines. Três desses riscos são cruciais para a arquitetura de DevSecOps.

Insufficient Credential Hygiene (Insuficiência de Higiene de Credenciais)

Este é o risco central da prática desta aula. Credenciais estáticas de longa duração, como as chaves de acesso AWS, representam um ponto único de falha. Se o repositório, o *runner* ou um log forem comprometidos, o atacante obtém acesso persistente com privilégios de longa duração. Embora provedores de nuvem como a AWS monitorem ativamente *commits* públicos em busca dessas chaves, essa defesa não é suficiente, pois a janela de exploração permanece aberta se o atacante for rápido. A mitigação fundamental é a eliminação total de chaves de longa duração.

Poisoned Pipeline Execution (PPE - Execução de Pipeline Envenenado)

O PPE ocorre quando um código malicioso é introduzido no repositório (por exemplo, um *script* alterado ou um novo *Makefile*) e, subsequentemente, executado pelo *runner* com privilégios elevados. Uma vez que o *runner* executa o código, o atacante pode usar os privilégios da máquina para exfiltrar segredos, comprometer a infraestrutura interna ou realizar outras atividades maliciosas.

A mitigação contra o PPE exige a separação de privilégios e o **isolamento de execução**. *Workflows* que processam código de fontes externas ou não revisadas (como *pull requests* de *forks*) devem ser executados em ambientes de *runner* com privilégio zero, isolados da rede interna e de segredos de produção. Além disso, controles rigorosos de SCM, como a exigência de revisão de *pull requests* e o uso de *branches* protegidas, são essenciais para prevenir a introdução de código malicioso.

Inadequate Identity and Access Management (Gerenciamento de Identidade e Acesso Inadequado)

Este risco é a falha na implementação rigorosa do **Princípio do Menor Privilégio** (*Least Privilege*). As permissões concedidas ao pipeline devem ser estritamente limitadas apenas às ações e recursos necessários para o *job*. O uso de permissões *curinga* (como *:** em políticas IAM) configura a vulnerabilidade, pois um comprometimento da identidade do pipeline resultaria em um *blast radius* inaceitável, permitindo ao atacante realizar ações destrutivas muito além do escopo de um *deploy* normal.

Arquitetura de Identidade: OpenID Connect (OIDC) e Credenciais Temporárias

O OpenID Connect (OIDC) é a resposta arquitetural moderna para mitigar o risco de credenciais estáticas (CICD-SEC-6). Ele viabiliza a **Federação de Identidade**, transformando a confiança entre o GitHub Actions e a AWS em um relacionamento gerenciado.

O grande benefício operacional do OIDC para o profissional sênior é a **eliminação da sobrecarga de rotação periódica de chaves** (*Zero Rotation Burden*), tornando a segurança um recurso arquitetural automatizado.

No fluxo OIDC, o *workflow* do GitHub Actions solicita um *JSON Web Token* (JWT) assinado. A AWS, que foi previamente configurada para confiar no emissor OIDC do GitHub (o *Identity Provider*), recebe este JWT e verifica sua autenticidade. Se a validação for bem-sucedida, a AWS utiliza seu *Security Token Service* (STS) para emitir uma sessão temporária de curta duração (*short-lived credentials*) via *AssumeRole*.

A segurança é otimizada pela implementação do Menor Privilégio dentro da política de confiança do IAM Role. É fundamental usar as chaves de condição (*claims*) do OIDC para validar a fonte, restringindo a Role para ser assumida **apenas** pelo repositório exato e **apenas** pela *branch* principal. Essa granularidade restringe o *blast radius* drasticamente: se um *fork* ou outra *branch* não autorizada for comprometida, o acesso aos recursos críticos de produção é automaticamente negado, fortalecendo a segurança do pipeline.

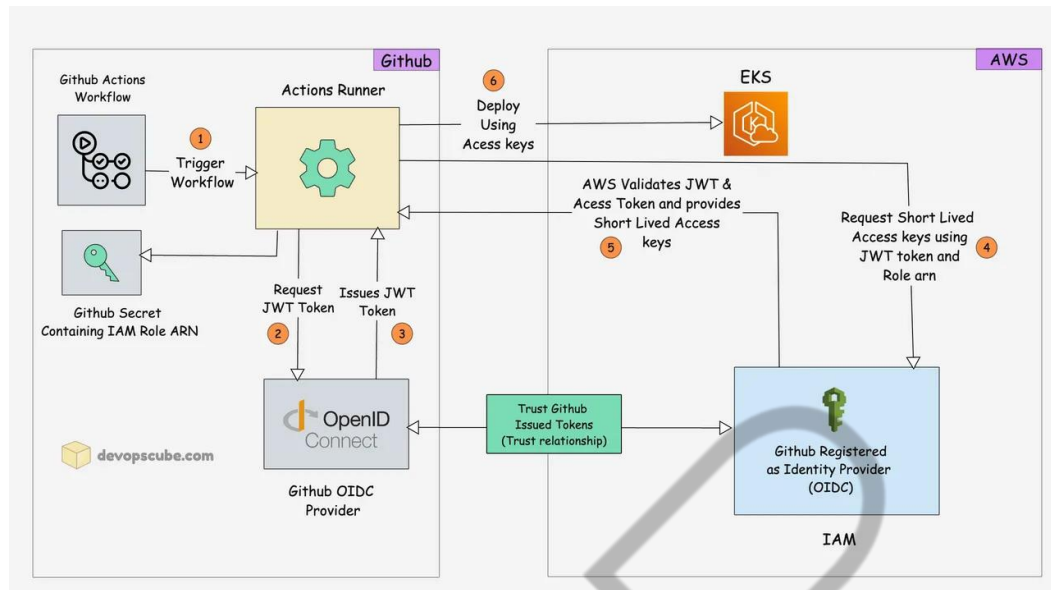


Figura 3 – Fluxo OpenID Connect - GitHub Actions e AWS

Fonte: [DevopsCube](https://devopscube.com) (2025)

Arquitetura de Execução: Comparativo entre SaaS e Self-Hosted Runners

O ambiente de execução do pipeline, o *runner*, é um recurso com privilégios. A escolha entre *GitHub-hosted runners* (SaaS) e *Self-hosted runners* (hospedados pelo cliente) impacta diretamente na arquitetura de segurança.

GitHub-Hosted Runners (SaaS)

Estes *runners* são máquinas virtuais efêmeras gerenciadas pelo GitHub. Sua principal vantagem de segurança é o isolamento: as VMs são *re-imaged* (limpas) após cada execução de *job*, garantindo que arquivos, dados sensíveis ou códigos maliciosos de execuções anteriores não persistam para comprometer trabalhos subsequentes. Eles oferecem uma segurança *built-in* e são adequados para a maioria das cargas de trabalho, exigindo apenas configuração mínima.

Self-Hosted Runners (AWS EC2)

Os *runners self-hosted* são essenciais para cenários de alta conformidade ou quando é necessário acessar recursos de rede privada (VPC) que não são acessíveis pela infraestrutura pública do GitHub.

No entanto, ao hospedar *runners* em sua própria infraestrutura (como instâncias AWS EC2), a responsabilidade de DevSecOps aumenta. O cliente é

responsável por manter o sistema operacional seguro, realizar *patching*, monitoramento e garantir o isolamento. O risco de **Execução de Pipeline Envenenado (PPE)** se agrava aqui, pois o comprometimento de um *runner* EC2 pode conceder ao atacante acesso ao **IAM Role de Instância** (*Instance Profile*), que potencialmente comprometeria recursos na rede interna da VPC.

Para mitigar esses riscos, a arquitetura deve ser rigorosa:

1. Utilizar **IAM Roles para EC2** para delegar permissões, em vez de chaves estáticas (eliminando o risco de armazenamento de chaves na própria máquina).
2. Implementar **isolamento de rede** estrito, garantindo que o *runner* acesse apenas os *endpoints* necessários.
3. Utilizar *runners* **efêmeros** e descartáveis, provisionados sob demanda e destruídos após o *job*. Este método previne a persistência de ameaças no ambiente de execução.

MERCADO, CASES E TENDÊNCIAS

A consolidação do DevSecOps não é apenas uma boa prática técnica; tornou-se a espinha dorsal de qualquer estratégia de negócio *cloud-native*. No mercado global, a integração de segurança desde o início do ciclo de vida é um requisito fundamental para a sobrevivência em um cenário de ameaças em rápida evolução.

Uma tendência crucial que redefine o futuro do CI/CD e do DevSecOps é a ascensão da **Engenharia de Plataformas** (*Platform Engineering*). Esta disciplina surge como a evolução do DevOps, focando na construção de **Plataformas Internas de Desenvolvedor (IDPs)** de autoatendimento. O objetivo dessas plataformas é padronizar o fluxo de trabalho e abstrair a complexidade do *toolchain* de CI/CD e da infraestrutura subjacente para as equipes de produto.

A IDP é o veículo mais eficaz para operacionalizar o *Shift-Left*. Em vez de exigir que cada time de desenvolvimento configure manualmente as ferramentas de segurança (SAST, OIDC, *secrets checking*), a Plataforma de Engenharia fornece *golden templates* e *blueprints* de CI/CD que incorporam, por padrão, as melhores práticas de DevSecOps. Por exemplo, todos os *templates* de *deploy* vêm pré-configurados para utilizar o OpenID Connect (OIDC) para credenciais temporárias, e incluem regras de *Policy-as-Code* que garantem o menor privilégio.

O sucesso desta abordagem reside em fazer com que o caminho seguro seja, arquiteturalmente, o caminho de menor resistência para o desenvolvedor. Isso acelera a produtividade em escala e garante que a arquitetura DevSecOps seja adotada organicamente, sem impor *overhead* ou fricção desnecessária na engenharia, permitindo que pessoas desenvolvedoras foquem exclusivamente na inovação e na entrega de valor.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, a fronteira de confiança crítica foi estabelecida no pipeline de CI/CD. Você compreendeu o imperativo econômico do **Shift-Left**, reconhecendo que corrigir falhas de segurança nas fases iniciais é uma estratégia de otimização de custo e produtividade.

Foi apresentada a estrutura **STRIDE** para a Modelagem de Ameaças em seu pipeline, capacitando-o a mapear sistematicamente riscos. A demonstração prática das **Credenciais Estáticas** motivou o estudo da migração arquitetural para o **OpenID Connect (OIDC)**, a solução de Zero Trust que fornece credenciais temporárias, implementando o princípio do **Menor Privilégio**.

Por fim, exploramos as implicações de segurança dos *Runners* (SaaS vs. Self-Hosted), destacando a responsabilidade inerente ao uso de instâncias EC2 e a necessidade de isolamento de execução. A discussão se encerrou com a análise da **Engenharia de Plataformas** como a tendência de mercado que transforma o DevSecOps em um recurso de autoatendimento e de fricção zero.

REFERÊNCIAS

ADKINS, HEATHER. **Building Secure and Reliable Systems: Best Practices for Designing, Implementing, and Maintaining Systems**. Sebastopol: O'Reilly Media, 2020.

CHECKPOINT. **What is Shift-Left Security?**. 2025. Disponível em: <https://www.checkpoint.com/cyber-hub/cloud-security/what-is-shift-left-security/>.

Acesso em: 19 nov. 2025.

GITHUB. **Configuring OpenID Connect in Amazon Web Services**. 2025. Disponível em: <https://docs.github.com/actions/deployment/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services>.

Acesso em: 28 nov. 2025.

KLEIN, C. **STRIDE Threat Model: A Complete Guide**. 2025. Disponível em: <https://www.jit.io/resources/app-security/stride-threat-model-a-complete-guide>.

Acesso em: 28 nov. 2025.

OWASP. **CI/CD Security Cheat Sheet**. 2025. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/CI_CD_Security_Cheat_Sheet.html. Acesso em: 28 nov. 2025.

PALAVRAS-CHAVE

DevSecOps. Shift-Left. CI/CD Security. OWASP CI/CD Top 10. OpenID Connect (OIDC). Credenciais Temporárias. IAM. GitHub Actions. Modelagem de Ameaças. STRIDE. Poisoned Pipeline Execution (PPE). Least Privilege. SaaS Runners. Self-Hosted Runners. Platform Engineering. Segurança na Nuvem.

EMANDA



POS TECH